Stream Assembly Is an Uncontrolled Treatment in Streaming Intrusion-Detection Benchmarks

MICHEL YOUSSEF, Independent Researcher, Lebanon

Public network captures are rarely usable as evaluation streams as they stand, so streaming intrusion-detection studies assemble them: interleaving capture days, pooling temporally disjoint captures, or replaying records round robin. We show on two benchmarks that this assembly step is not neutral plumbing but an *uncontrolled experimental treatment*. On CICIDS2017, holding the full record multiset identical and changing only the ordering, a fixed positional 70/15/15 split then produces held-out samples that share only 32.5% of their records, at held-out prevalences of 68.235% and 25.2396% — a 42.9954-point difference — and the measured ordering of the two deterministic scorers reverses. Restricting both arms to the 78000 records they both held out removes that reversal: the same scorer leads in both arms there. The reversal is therefore attributable to which records the assembly hands to the test set, not to the order in which the detector saw its history. That attribution assumes that history contributes no more on the records the arms do not share than on those they do. On LITNET-2020, pooling three temporally disjoint captures reports a single 6.4982% operating point that is the equal-weight mean of per-capture held-out prevalences spanning 0.176% to 15.7747%; we present that identity as an audit check rather than a discovery. We also audit the evaluated detector against its description: its reset and growth branches share a predictive term that cancels, so the reset posterior $P(r_t{=}0)$ equals the hazard rate exactly below the run-length cap, while the evaluations spend nearly all their length at or beyond that cap, where the posterior instead wanders; and the score the evaluation consumes is a function of $P(r \leq 5)$, not of $P(r{=}0)$. Scoring that detector one branch at a time yields a separate result: its deployed max composition ranks *worse than its own tail term alone* (0.103477 *AP*, 0.302658 AUC-ROC), because the auxiliary branch is inverted rather than uninformative (AUC-ROC 0.281890) and a maximum lets it set the record's score wherever the tail is small — a defect no metric computed on the assembled score can attribute. Finally, we quantify a batch dependence in the ECOD reference implementation, whose empirical CDFs are recomputed over the training matrix concatenated with the scored batch: holding the evaluated records and the fitted model fixed and changing only the accompanying batch moves its AUC-PR by 0.003063, so published ECOD numbers are not comparable across studies that score a different batch, in size or composition. Every measured value traces to an archived, hash-verified run manifest, and the sentence-level claim ledger ships with the artifact.

CCS Concepts: • **Security and privacy** → **Intrusion detection systems**; • **Computing methodologies** → *Machine learning*; • **General and reference** → *Evaluation*.

Additional Key Words and Phrases: streaming network intrusion detection, benchmark stream construction, evaluation methodology, experimental design, anomaly detection benchmarks, reproducibility, provenance

1 Introduction

Machine-learning evaluations for network intrusion detection rest on a small number of public benchmarks, and a substantial literature questions what those benchmarks measure [4, 15, 43]. This paper contributes a quantified case study and mechanism audit of one step in that pipeline: the step that assembles capture files into an evaluation stream. CICIDS2017 spans five capture days of very different attack density [20, 45], while LITNET-2020 annotates twelve attack types collected over about ten months [16] and the three captures we evaluate are temporally disjoint, so evaluations interleave, pool, or replay records to obtain a single stream with workable splits. We measure what that step does on these two benchmarks.

Author's Contact Information: Michel Youssef, Independent Researcher, Beirut, Lebanon, michelyoussef@hotmail.com.

What this paper claims, and what it does not

Our CICIDS2017 intervention changes the ordering of a fixed record multiset and holds the split *rule* fixed. It does not hold the evaluated *sample* fixed: under a positional split, reordering changes which records land in training, validation and test, and therefore changes held-out membership, held-out prevalence, and within-split order simultaneously. Stream assembly is in this precise sense an **uncontrolled treatment**. We therefore make a pipeline-level claim — assembling the stream differently changes the measured evaluation outcome — and we do *not* claim that order alone causes the change, nor that attack prevalence is excluded as an explanation. Isolating those factors requires a factorial or prevalence-matched design that we identify as the natural next experiment.

Two further scoping statements belong here rather than in a limitations section. First, we make no claim about how often the practice occurs: we have not conducted a systematic literature survey, and the construction we contrast against is the one used by the audited earlier evaluation of this same work, with interleaving and pooling also discussed in the cited benchmark-criticism literature. Second, the conceptual warning that arbitrary temporal ordering and sampling can distort security-ML evaluation is established prior art [4, 15]; our contribution is the exact same-multiset measurement on a real benchmark, with overlap, relocation, prevalence and outcome reported together, plus the audit of the released detector.

Origin of this version

Earlier versions of this manuscript (arXiv:2605.24696, v1 and v2) reported results produced under the pooled and interleaved constructions studied here, and described a scoring rule the released code did not implement. An independent adversarial review and a line-by-line audit of the archived artifacts established both. The present version is the rebuild from that audit: the assembled constructions are retained as explicitly labelled protocols and contrasted against least-transformed alternatives; the detector is described as what the code computes; and every number was regenerated — or, for the retained prevalence sweep, re-derived from the archived pre-audit runs — by a script that wrote, in the same execution, an archived manifest recording the git commit, configuration hash, input SHA-256, seed, environment hash and output path. Section 11 states which measurements this paper shares with those versions and which are new; the per-result-group account and the dated correction history are supplied to the editors confidentially, because printing them here would defeat anonymization.

Contributions

(1) **Stream assembly as an uncontrolled treatment** (Section 5). On the identical 1600000-record multiset (352962 attacks, whole-stream prevalence 22.060125%, permutation verified mechanically), reordering under a fixed positional split yields held-out slices sharing 78000 of 240000 records (32.5%), moves held-out prevalence by 42.9954 points, relocates 103189 attacks out of the held-out slice, and reverses the measured ordering of the two deterministic scorers — a reversal that does *not* survive when both arms are restricted to the records they both held out (Section 6).

(2) **A pooling identity, offered as an audit check** (Section 5.4). LITNET's pooled 6.4982% is the equal-weight mean of the three per-capture held-out prevalences, forced by equal budgets and a divisible round-robin boundary. It is arithmetic, not a discovery; its value is diagnostic — a pooled operating point can correspond to no capture in the pool.

(3) **A scoped method-identity audit** (Section 3). Below the run-length cap the reset and growth branches share a predictive term that cancels, so the reset posterior $P(r_t{=}0)$ equals the hazard rate for any data, a property of the published recursion [1] and not a coding error; at and beyond the cap — where the evaluated runs spend nearly all their length — truncation breaks that identity and the posterior wanders. The score the evaluation consumes is a function of $P(r_t \leq 5)$, not the $P(r_t{=}0)$ the earlier description defined it as. We state these three facts separately because only the first is an exact result.

(4) **A composition defect that costs more than it appears to** (Section 7). Scored branch-wise on the timestamp-ordered held-out slice, the deployed max composition ranks *worse than its own tail term alone* — by 0.103477 *AP* and 0.302658 AUC-ROC. The auxiliary branch is not uninformative but inverted (AUC-ROC 0.281890), and because the score is a maximum it sets the record's score wherever the tail is small, dragging the deployed detector to near-chance AUC-ROC (0.526623) while its tail component reaches 0.829281. No metric computed on the assembled score can attribute this, and the decomposition that finds it uses values the scorer already computes.

(5) **A measured batch dependence in a widely used baseline** (Section 6.3). PyOD's ECOD recomputes its empirical CDFs over the training matrix concatenated with whatever batch is passed to `decision_function`, so a record's score depends on which other records accompanied it. Holding the evaluated records and the fitted model identical and varying only the accompanying batch moves ECOD's AUC-PR by 0.003063. Published ECOD numbers are therefore not comparable across studies that score a different batch, in size or composition.

(6) **A provenance discipline that is itself auditable** (Section 10), including the correction history and overlap statement of Section 11.

2 Related Work

Benchmark criticism. Catillo et al. [15] question whether results on public intrusion datasets represent concrete advances and warn specifically that partitioning timestamped records in arbitrary order can alter split representativeness. Arp et al. [4] catalogue sampling bias, mixing incompatible sources and temporal snooping as recurring security-ML pitfalls. Sommer and Paxson [46] set out earlier why machine learning is harder to apply to intrusion detection than to other domains, pointing to the high cost of errors, the semantic gap between results and operational meaning, and the difficulty of sound evaluation. Ring et al. [43] survey dataset limitations, and Engelen et al. [20] correct label and flow-construction defects in CICIDS2017; we build on their corrected release. Liu et al. [33] quantify how prevalent such labelling errors are in CIC-IDS-2017 and CSE-CIC-IDS-2018, and Lanvin et al. [29] show that correcting the CICIDS2017 errors changes measured detection performance. Catillo et al. [14] earlier showed that a detector learned on a public IDS dataset degrades against emulated attacks in a controlled network, which is the transfer-side counterpart of the question we ask on the construction side. We do not claim to discover that temporal assembly is non-neutral. Our addition is quantitative and specific: the same full multiset under two assemblies, with membership overlap, attack relocation, prevalence and measured outcome reported together on a real benchmark.

The evaluation protocol as the variable under study. A line of work treats the protocol itself, rather than the model, as what determines a reported result. TESSERACT [41] shows that spatial and temporal bias in how train/test sets are constituted inflates malware classification results, and gives a construction discipline for removing it; Apruzzese et al. [3] formalise cross-evaluation and show how much a reported NIDS result depends on which corpora are combined; Olszewski et al. [40] find that a large fraction of tier-1 security ML papers cannot be reproduced from what they report. Apruzzese, Laskov and Schneider [2] survey how machine-learning NIDS are assessed and set out pragmatic criteria for evaluations that practitioners can act on. Concurrent with this work, Moczkodan and Ragab [39] reformulate CICIDS2017 as a temporal task and find that a padding convention, not the architecture, determines whether Transformers appear to help — a protocol variable of a different kind (input formatting) on the same benchmark. Biswas [10] likewise argues the evaluation protocol is the hidden variable: within the single corpus CSE-CIC-IDS2018 it compares random, stratified-temporal and class-blind chronological splits, uses the stratified-temporal arm to attribute the chronological collapse to attack families unseen in training rather than to drift within known classes, reports zero-shot transfer to CIC-DDoS2019 as a separate experiment, and scores each protocol on its own held-out partition rather than on records held out by both. This work varies the assembly of one fixed record multiset and attributes the outcome change to *composition* rather than to *history order* by restricting both arms to the records they both held out. The two are complementary: theirs separates unseen-attack coverage from drift within one corpus and reports transfer between corpora separately, ours attributes an assembly effect within one corpus under a stated assumption.

Streaming anomaly detection. The baselines evaluated in this rebuild are Half-Space Trees [47] and LODA [42] (streaming), with LOF [11] and ECOD [32] as batch diagnostic references. KitNET [38], xStream [35], RRCF [23], streaming isolation forest [18] and COPOD [31] have implementations in the artifact but no archived evaluation runs in this rebuild, and carry no numbers here. Cao et al. [13] benchmark streaming detectors under controlled anomaly types and drifts and report strong sensitivity to assessment design; our results give one such sensitivity a measured mechanism on an IDS benchmark. Wu and Keogh [51] argue that the standard time-series anomaly benchmarks are themselves flawed in ways that create an illusion of progress.

Change-point detection. The evaluated detector descends from Bayesian online change-point detection [1] with a truncated run-length posterior [21, 27]. Section 3 shows that the reset posterior of the published recursion, which the implementation reproduces, is data-independent below the cap; van den Burg and Williams [49] document how often change-point evaluations mismeasure their subject.

Conformal and calibrated alerting. Laxhammar and Falkman [30] established conformal anomaly detection for sequential data, with guarantees resting on exchangeability assumptions [50]. Three 2026 lines run concurrent with this work and are distinct from it. FIRCE [8] and its successor FADES [7] integrate conformal evaluation into streaming IDS pipelines to drive drift detection and adaptive retraining; both propose detection machinery and evaluate it, whereas this paper proposes no detector and instead audits what an evaluation pipeline measures. Gurjar and Camp [24] train a gradient-boosted classifier on smoothed alert intensity, momentum and volatility features to predict whether alert intensity will exceed the training period's 95th percentile within the next 30 minutes, fitted per severity class on 251 million Suricata alerts from one university network — a forecasting task on operational alert volume, not a question about how a benchmark is assembled. None of the three examines the assembly step. Our results are relevant but must be stated carefully: reordering one realized dataset does not establish exchangeability, which is a distributional symmetry. What interleaving can do is *mask temporal dependence, or make exchangeability appear more plausible to a diagnostic* applied to the assembled stream. Conformal methods that relax exchangeability already exist [6].

Metric behaviour under class imbalance. That precision-recall summaries move with the positive-class base rate is long-established and is not a finding of this paper. Davis and Goadrich [17] characterise the ROC/PR correspondence and show that the achievable PR region depends on the class ratio; Saito and Rehmsmeier [44] demonstrate that PR summaries are the informative choice precisely because they respond to imbalance while ROC summaries do not; and Axelsson [5] established the base-rate consequence for intrusion detection specifically. McDermott et al. [37] caution that AUC-PR is not automatically the better summary under class imbalance and that the choice should follow how the scores will be used. We use these results, we do not extend them: every AUC-PR this paper compares *across* prevalence regimes is reported with its chance level p and with normalized lift $(AP - p)/(1 - p)$ for that reason, and comparisons made within one slice, where the chance level is shared, are reported at that fixed level. What is ours is narrower and is about this benchmark's construction, not about the metric: the prevalence regime these evaluations operate in is *doubly constructed* — first by a retention budget whose per-day sampling is anti-correlated with attack density, and then by the assembly step, which redistributes the surviving attacks across the split. Neither construction step is visible in a reported operating point.

Cost-sensitive thresholds. The threshold actually implemented is the prior-inclusive cutoff formula of Elkan [19] (Section 3); SRE-style error budgets [9] motivated the original framing.

3 The System Under Study

We characterize the detector as implemented, from measurement. All values here are archived in the artifact.

3.1 The score the evaluation consumes

The published description defines the anomaly score as the run-length-reset posterior $P(r_t{=}0 \mid x_{1:t})$. The implementation returns

$$s_t \;=\; \max\big(\mathrm{tail}_t,\; 0.25 \cdot P(r_t \leq 5 \mid x_{1:t})\big),$$

where tail_t is a χ^2 CDF of the squared standardized residual under a global, slowly adapting diagonal Gaussian. Two facts follow directly. The evaluated score is not the described score; and it is a function of $P(r_t \leq 5)$, so results about $P(r_t{=}0)$ do not transfer to it without further argument. Because $0.25 \cdot P(r \leq 5) \leq 0.25$, the auxiliary branch can set the score only where the tail term is below 0.25. Measured over the 49970 post-warm-up records at the *head* of the timestamp-ordered CICIDS2017 stream — a prefix, not the whole stream, and one that carries no attacks at all — it does so on 75.039% of records, at a mean contributed value of 0.002531. The same share measured on the attack-bearing held-out slice is 58.452% (Section 7); the two are different populations and are not in tension. A mean magnitude does not establish which branch *discriminates*, since ranking metrics are invariant to the scale of a branch that sets the ranking; we therefore make no claim about that here and measure it directly in Section 7.

3.2 The run-length posterior below the cap, and at it

In the Adams and MacKay recursion as published [1] (their Eq. 3 and Algorithm 1), the changepoint and growth branches at step t carry the same run-conditional predictive term, which cancels in the posterior normalization: under a constant hazard the reset posterior $P(r_t{=}0 \mid x_{1:t})$ equals the hazard rate h *exactly, for any data*, because the reset hypothesis at t is not informed by x_t and a change shows in the posterior one step later, on $r_{t+1}{=}1$. The implementation reproduces that recursion below the run-length cap, so this is a property of the algorithm, not a coding error. The audit finding is narrower: the earlier description defined the score as that reset posterior, which is uninformative below the cap by construction, whereas the implemented score is the max composition over $P(r_t \leq 5)$ stated above. Under a 6σ mean shift with hazard 0.001: mean $P(r{=}0)$ before the shift 0.001; peak in the 50-record window after it 0.001; maximum deviation from the hazard between initialization and truncation 0.

That exact result covers only part of the evaluated regime, and we state the limit plainly. At $t \geq 500$ the run array is truncated before normalization, the cancellation no longer holds, and the posterior wanders (mean 0.01099, maximum 1). Every stream evaluated in this paper is far longer than 500 records, so the published runs spent nearly all of their length in the truncated regime. We have therefore proved data-independence where it is provable and measured behaviour

Table 1. The evaluation streams (Stage 1 manifests). Run lengths are contiguous attack runs in the labelled stream; span is the wall-clock extent of the capture.

stream	records	prevalence	held-out prev.	run med/p90/max	span (min)
CICIDS2017 (week)	1600000	22.0601%	68.235%	2/70/2522	6246.71
LITNET udp_flood	500000	14.9384%	15.7747%	1/2/20	3.97
LITNET blaster_worm	500000	0.6562%	3.544%	1/2/5	1.62
LITNET spam	500000	0.0276%	0.176%	1/1/2	39122.23

where it is not; we have not established that the auxiliary branch is uninformative over the regime that produced almost all reported scores, and we do not claim it.

3.3 The threshold

The implemented decision cutoff is the prior-inclusive formula of Elkan [19], $\tau = C_{FP}(1-\rho)/[C_{FP}(1-\rho)+C_{FN}\rho]$, giving 0.655172 at incident prior ρ=0.05 and 0.261745 at ρ=0.22 — not the cost-only $C_{FP}/(C_{FP}+C_{FN})$ = 0.090909 described in earlier versions. That formula is a Bayes rule for a calibrated class posterior. The quantity it is applied to here is the max composition defined at the head of this section, which is not one, so the cutoff carries no Bayes-optimality warrant on this score. We record that as an observation about what the implementation does and not as a defect claim, and it changes no reported number: AUC-PR and AUC-ROC are threshold-free and unaffected; we report no threshold-dependent column, which is also why none of the non-comparable precision/recall/F1 columns appears in this paper.

3.4 Detection delay, not latency

The quantity previously reported as “latency in milliseconds” is a count of records between attack onset and first alert; the evaluation contains no wall-clock instrumentation (0 clock calls in the module). We report it as detection delay in records and report no per-flow compute cost, which this pipeline does not measure.

4 Datasets, Assembly, and Protocol

4.1 CICIDS2017: a timestamp-sorted budgeted subsample

We use the Engelen-corrected release [20]. The evaluation corpus is a **timestamp-sorted budgeted subsample**: 1600000 of 2087997 eligible records, a retention of 76.628%, selected by fixed per-day budgets with proportional stride subsampling. Because the budgets are fixed rather than a common factor, larger days are cut harder and the attack-heavy days are the largest: per-day retention runs from 64.399% (Friday, the densest day) to 93.156% (Tuesday). Measured consequence: the eligible week’s prevalence is 24.2065% while the corpus is 22.0601%, a bias of -2.1464 points from the budget step alone. Sorting a stride-subsampled corpus preserves relative order but is not the full event stream; missing flows can break attack runs and alter online sufficient statistics. We measured the prevalence bias and did not measure the effect on attack-family mix, feature distributions or run lengths, so we use “timestamp-sorted budgeted subsample” throughout and avoid claims that require the complete natural chronology. Table 1 summarizes all four streams.

4.2 LITNET-2020: three disjoint attack-type captures

The three captures this paper evaluates have no common chronology: they do not overlap in time, they are separated by gaps of weeks and months, and their spans are 3.97, 1.62 and 39122.23 minutes (Table 1), so a global timestamp order over them would concatenate three intervals across those gaps rather than describe one observed stream. We make no chronology claim about LITNET-2020 as a whole, which was collected over about ten months and annotates twelve attack types [16]. We evaluate three per-attack-type streams of 500000 records each, never a manufactured composite; the pooled composite appears only as the explicitly labelled assembled arm of Section 5.4. Multi-window burn-rate alerting over 60/360/4320-minute windows is undefined on the two captures that span minutes; we report that as a finding about the benchmark, and no burn-rate result appears in this paper.

4.3 Protocol

Tables 2 and 3 state the protocol in full so the paper is self-contained; the artifact reproduces it exactly.
Stream assembly, stated as an algorithm:

```
timestamp-order arm:
  rows <- corpus sorted by Timestamp (stable mergesort)
day-round-robin arm:
  groups <- rows grouped by capture date, each in timestamp order
  for k = 0, 1, 2, ...:
      for d in sorted(dates):
          if k < len(groups[d]): emit groups[d][k]
pooled-composite arm (LITNET):
  groups <- per-attack-type streams, each in timestamp order
  emit round robin over sorted(attack_type) as above
split (both arms): i_tr = floor(0.70 n); i_va = i_tr + floor(0.15 n)
```

5 Assembly as a Treatment

5.1 Design and what it identifies

CICIDS2017 admits a clean manipulation of the assembly step: the same record multiset can be presented in timestamp order or in day-of-week round robin. The reordering is verified mechanically to be an exact permutation — both arms hold 1600000 records and 352962 attacks (22.060125%). The assembled arm reproduces the archived earlier evaluation exactly (60575 held-out attacks in 240000 records), so the contrast is against the assembly actually used, not a strawman.

What the design does *not* do is hold the evaluated sample fixed. Under a positional split, reordering changes split membership, so the two arms differ in held-out records, held-out prevalence, training composition and within-split order at once. The estimand is therefore the effect of the complete assembly pipeline under a fixed split rule, and nothing finer. We report the joint change and decline to decompose it. Figure 1 shows why the held-out membership differs.

5.2 What assembly does to the evaluated sample

Held-out prevalence differs by 42.9954 points. The two held-out slices share only 78000 of 240000 records (32.5%); the assembled arm's held-out attacks are a strict subset of the timestamp-ordered arm's, with 103189 attacks relocated out of the held-out slice: 77670 of them into training and 25519 into validation, counted from the archived splits. The mechanism of the prevalence change is dilution: every held-out attack in the assembled arm is a Friday record (1 of five days contribute any), while Monday–Thursday contribute 162000 attack-free held-out rows. The assembled arm's Friday sub-slice is denser (77.66%) than the timestamp-ordered arm's entire slice: prevalence falls because attack-free rows from the four other days enter the held-out slice and displace Friday attack rows, which is the same movement

Table 2. Experimental protocol. Every entry is taken from the code the archived manifests pin, not from intent.

element	setting
features	all numeric columns after dropping the label; non-numeric columns dropped, so no categorical encoding is used; d = 84 (CICIDS2017), 36 (udp_flood), 36 (blaster_worm), 36 (spam)
scaling	none applied by the harness
missing / infinite	$\pm\infty$ mapped to NaN, then all NaN filled with 0.0; this touches 4 of the 1600000 CICIDS2017 rows (8 infinite cells in 2 features, no NaN) and 0, 0 and 0 rows of the three LITNET streams
label rule	y=1 unless the label string is `benign`, `normal`, `0` or `false`
eligibility (CICIDS)	rows whose label contains " - Attempted" dropped
per-day budgets	300k / 300k / 350k / 300k / 350k (Mon–Fri)
subsampling	proportional stride within each class, preserving per-day prevalence
split	positional 70/15/15 over the assembled stream order
detector: hazard	0.001
detector: run-length cap	500
detector: warm-up	30 records
detector: short-run mass	$P(r \leq 5)$, weight 0.25
detector: predictive model	one diagonal Gaussian over all d features, initialised on the first record, with mean and M_2 advanced by Welford's recurrence after every record and variances floored at 10^{-4}; the tail score is the χ^2_d CDF of the squared standardized residual under the pre-update global statistics, zero during warm-up; run-conditional means and M_2 are kept per run length by the same recurrence
metric	`sklearn.metrics.average_precision_score` (AUC-PR) and `roc_auc_score`, scikit-learn 1.8.0, positive class 1, no sample weighting; average precision is the step-wise sum $\sum_n (R_n - R_{n-1}) P_n$, not trapezoidal interpolation. Flow-wise average precision is the convention of the compared work; event-based precision and recall for time series [48] are future work
chance level	the expected average precision of an uninformative ranking is taken as the positive-class prevalence p; the exact finite-sample expectation of step-wise AP under a uniformly random permutation differs from p in the fifth decimal (0.682366 against 0.682350 on the timestamp-order held-out slice; the full-list case of Theorem 1 of Manzhos et al. [34]) and changes no conclusion here; it is an expectation, not a lower bound, and a ranking can fall below it
reported precision	every *dimensionless* metric on $[0, 1]$ — AUC-PR, AUC-ROC, chance level, fractional prevalence, normalized and additive lift, and the margins, ranges and differences taken between them, including per-seed draws — is reported to six decimal places, and any of those derived from other reported quantities is computed from their *reported* values. Every such derived number therefore equals the arithmetic on the numbers printed beside it, exactly, and a reader can check any of them; the cost is that it may differ from its full-precision counterpart by a few units in the last reported place, below 10^{-5} throughout and orders of magnitude below any effect claimed here. Deliberately outside that rule, and reported at the precision each was emitted with: percentage- and percentage-point-scaled values (the same quantities at 100×, where six decimals would fabricate digits), integer counts, the protocol constants in this table and in Table 3, the posterior probabilities of Section 3 and the diagnostic fractions of Table 10, which are not metrics, and values quoted from cited work

Table 3. Experimental protocol, continued: update timing, label access, baseline hyperparameters, seeds and the sweep's resampling procedure. Every entry is read from the code the archived manifests pin.

element	setting
update timing	prequential, in the sense of Gama et al. [22]: score computed from the pre-update predictive; sufficient statistics updated after scoring, on every record including held-out ones
label access	the detector and HST see no labels at any point; baselines receive a validation-derived threshold; **ECOD and LOF are fitted on benign-only training rows and therefore have label access**
decision threshold	the evaluated detector alerts at Elkan's prior-inclusive cutoff $\tau = C_{FP}(1-\rho)/[C_{FP}(1-\rho) + C_{FN}\rho]$ with $C_{FP} = 1$, $C_{FN} = 10$ and $\rho = 0.05$ in the contrast runs (0.655172); each baseline alerts at the threshold that maximises F1 on the validation slice's precision–recall curve, falling back to the same τ when validation holds one class; AUC-PR and AUC-ROC are threshold-free and no threshold-dependent column is reported
HST	river `HalfSpaceTrees` behind a river `MinMaxScaler`: 25 trees, height 15, window 250, the cell's seed
LODA	the artifact's own implementation of LODA: 100 unit-norm random projections, 32 histogram bins, window 512, running standardization, the cell's seed
LOF	`sklearn.neighbors.LocalOutlierFactor`, 35 neighbours, `novelty=True`, `contamination='auto'`, fitted on benign-only training rows; score = −`score_samples`, min–max scaled
ECOD	PyOD 2.0.5 `ECOD()` at its defaults, fitted on benign-only training rows and scored in one `decision_function` call on the batch stated in Section 6.3; scores min–max scaled
seeds	contrast and LITNET runs: seed 11, with HST's additional draws at seeds 23 and 47; sweep: resample seeds 11, 23 and 47, each streaming method seeded with the resample seed
sweep resampling	stratified within each chronological split segment: below the natural rate every benign row is kept and attacks are thinned to the segment quota; above it both classes are thinned to the quotas of the largest stream whose re-split meets the target in every segment; selection is random without replacement, order preserved, no duplicates; a draw is accepted only if every segment's achieved prevalence is within 1 pp of the target, otherwise redrawn with seed + 100000·attempt, at most 20 redraws

Table 4. The assembly contrast. Held-out prevalence, attacks, and the evaluated detector's AUC-PR per arm. Every cell is a macro resolving to an archived manifest.

Benchmark	Construction	Held-out prev.	Attacks	AUC-PR (evaluated detector)
CICIDS2017	natural (timestamp)	68.235%	163764	0.728337
CICIDS2017	synthetic (day RR)	25.2396%	60575	0.544998
LITNET udp flood	natural (per type)	15.7747%	11831	0.394428
LITNET blaster worm	natural (per type)	3.544%	2658	0.964091
LITNET spam	natural (per type)	0.176%	132	0.846154
LITNET pooled	synthetic (3-type RR)	6.4982%	14621	0.943056

the relocation counts above report. This dilution accounts for the prevalence change; it is not by itself an account of the outcome change in Section 5.3.

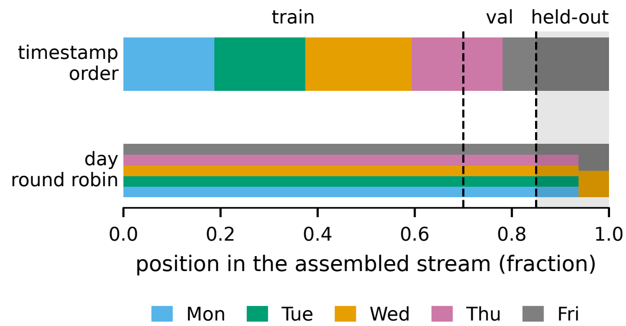

Fig. 1. The two assemblies of the same CICIDS2017 records under one fixed positional split. In timestamp order the capture days follow one another, so the held-out slice is the tail of the last day. In day round robin every day still supplying records contributes to every position, so the held-out slice draws from the tail of each day. Block widths follow the per-day record budgets of Table 2. The figure is a schematic of the protocol and carries no measured value.

5.3 What assembly does to the measured outcome

AUC-PR's chance level is the positive-class prevalence, which the treatment moves (0.682350 against 0.252396), and additive lift $AP - p$ has a prevalence-dependent ceiling of $1 - p$. We therefore report raw AP, the chance level p, and the normalized lift $(AP - p)/(1 - p)$ together, and report no raw cross-arm difference.

- **Timestamp order** ($p = 0.682350$): ECOD $AP = 0.755142$ (normalized lift 0.229158), detector $AP = 0.728337$ (normalized lift 0.144773). HST scores 0.588902 here, 0.093448 below its chance level, from a single seed, so no placement is asserted for it.
- **Day round robin** ($p = 0.252396$): detector $AP = 0.544998$ (normalized lift 0.391386), ECOD $AP = 0.418966$ (normalized lift 0.222805). HST's three-seed values are 0.513623, 0.427004 and 0.358504 (mean 0.433044, sd 0.077736); its ordering against ECOD flips across those seeds, so it carries no placement.

The measured ordering of the two deterministic scorers reverses between arms, in both raw and normalized terms. **This is a pipeline-level evaluation outcome under the stated protocol, not a statement about method quality.** Three qualifications are load-bearing and are stated here rather than deferred. (i) ECOD is fitted on benign-only training rows and therefore has label access the streaming methods do not; we treat it throughout as a *label-privileged diagnostic reference*, and no superiority is claimed for or against it. (ii) The two arms score different held-out samples, so the reversal is not measured on a common sample. Section 6 measures what happens when that difference is removed, and the reversal does not survive it (Figure 2). (iii) Determinism removes seed variance but not sampling uncertainty from the selected window, correlated attack runs, or the subsample. The margins are 0.026805 AP for ECOD in timestamp order and 0.126032 AP for the detector in day round robin, each computed from the reported cells; their event-block bootstrap intervals over the held-out slice (Section 12 states the procedure; the intervals are computed from the archived per-record scores) are [0.021099, 0.032692] and [0.122046, 0.130294], neither of which crosses zero. Those intervals cover sampling uncertainty over this slice only, so we claim no stable ordering, only a measured one under this protocol.

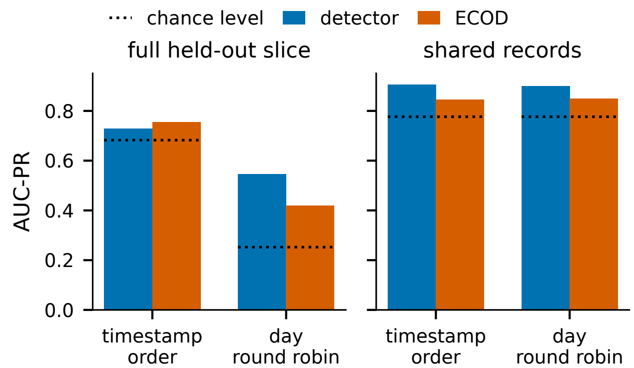

Fig. 2. Detector and ECOD AUC-PR in the two arms. Left: the full held-out slice of each arm, where the measured ordering reverses. Right: the 78000 records both arms held out, where the detector leads in both arms; the ECOD bars there are the slice-batch values of Table 6. Dotted marks are each group's chance level, the held-out prevalence: 0.682350 and 0.252396 for the full slices and 0.776603 for the shared records. Values are those of Section 5.3 and Table 6.

Table 5. LITNET-2020: AUC-PR per stream against the pooled assembled composite. The detector and ECOD (label-privileged diagnostic reference) are deterministic; HST is a single draw per cell, so no ordering involving HST is asserted. Its three composite draws across seeds are 0.238840, 0.177599, 0.367761.

stream	held-out prev.	detector	ECOD	HST (1 seed)
udp_flood (per type)	15.7747%	0.394428	0.321185	0.552675
blaster_worm (per type)	3.544%	0.964091	0.035710	0.147731
spam (per type)	0.176%	0.846154	0.106805	0.008447
pooled (assembled)	6.4982%	0.943056	0.229143	0.238840

5.4 LITNET-2020: pooling as an audit check

Round robin *within* a per-attack-type stream is the identity, so the LITNET contrast is compositional: three per-type streams against the pooled composite used by earlier versions. The pooled held-out slice is by construction exactly the union of the three per-type held-out slices — equal budgets, a perfect three-cycle, and a divisible split boundary — so its prevalence 6.4982% is the equal-weight mean of 15.7747%, 3.544% and 0.176%. This is arithmetic, and we present it as an audit check with pedagogical value rather than as a finding: a pooled operating point can correspond to no capture in the pool, and the equal weighting is an artifact of equal budgets. Per-stream values are in Table 5.

6 Isolating What the Treatment Changes

The contrast of Section 5 moves membership, prevalence and order together. Three analyses separate what can be separated from the archived per-record scores of both arms. All three use the metric implementation of Table 2; the dumps reproduce the archived arms to 0.000018 AUC-PR on the timestamp-ordered arm and exactly on the assembled arm.

Table 6. Both arms restricted to the 78000 records they both held out. Membership and prevalence are held fixed by construction; everything the assembly changes upstream of scoring still varies: the detector's prefix, ECOD's benign-only training rows and, in the slice columns, its batch. None of it reverses the ordering: the detector moves by 0.005242 *AP* between the two prefixes and ECOD by 0.002788 *AP* between the two fitted models on the identical shared batch. ECOD is reported under two scoring batches: "slice" is the arm's full 240000-record held-out slice, scored together and restricted afterwards to the shared records; "shared" is the 78000 shared records scored as their own batch under the same fitted model, which is the batch-controlled comparison (Section 6.3).

	AP			AUC-ROC		
arm	detector	ECOD (slice)	ECOD (shared)	detector	ECOD (slice)	ECOD (shared)
timestamp order	0.905613	0.844487	0.847585	0.869219	0.799910	0.802003
day round robin	0.900371	0.849842	0.844797	0.864159	0.805657	0.797670

6.1 The records both arms held out

The two held-out slices share 78000 records, carrying 60575 attacks at prevalence 0.776603 — they are the Friday records both assemblies happen to place in the test set, which is why their prevalence is far above either arm's. Restricting both arms to exactly those records, and recomputing:

The ordering does not reverse on the shared records, under either batch. The detector leads ECOD in both arms: by 0.061126 and 0.050529 *AP* under the slice batch and by 0.058028 and 0.055574 *AP* under the shared batch, each computed from the reported cells of Table 6, with event-block bootstrap intervals (Section 12) [0.053856, 0.067720], [0.044634, 0.056329], [0.051770, 0.064229] and [0.048620, 0.061923], none of which crosses zero. The detector's own score changes by only 0.005242 *AP* between the two histories, and ECOD's batch-controlled score by 0.002788 *AP* between the two fitted models. This is the paper's most consequential result and it constrains the paper's central claim: the reversal reported in Section 5.3 is attributable to *which records the assembly places in the test set*, not to the order in which the detector processed its history. That attribution rests on an assumption the archived design does not test: that history contributes no more on the records the arms do not share than on those they do. Stated exactly, the restriction shows that the ordering does not reverse on a sample both arms held out and that the detector's score on that sample moves by 0.005242 *AP* between the two histories; it does not bound history's effect on comparisons between shared and unshared records, which the full-slice *AP* of Section 5.3 also contains. We report the reversal as an evaluation-level consequence of assembly and explicitly not as an effect of ordering on detector behaviour.

Two cautions on reading Table 6. The shared records are not a random subsample — they are the intersection two particular assemblies produce, at a prevalence unlike either arm's — so its absolute values characterise that intersection, not the benchmark. And the two rows still differ in everything upstream of scoring, the detector's prefix and ECOD's benign-only training rows; what is held fixed is the evaluated sample, not the model state.

6.2 Where the split falls

A detector score depends only on the records preceding it, so one pass supports every chronological cut point; ECOD is refit at each cut because its training set moves with the boundary. Over 7 cuts from 60% to 90% of the stream, ECOD exceeds the detector at 3 of them, including the 85% cut this paper's fixed split uses, and the detector leads at the others. The measured ordering is therefore a property of the split point as well as of the assembly; the archived split is not adversarially chosen, but it is not neutral either, and no claim here is a stable property of CICIDS2017. Per-cut values are in Table 7 and Figure 3. Repeating the assembly contrast at the same 7 cuts, with both arms scored at each cut and ECOD refit at each, the two arms' orderings differ at 3 cuts — 75%, 80% and the 85% cut this paper uses, where ECOD

Table 7. The paired cut-by-assembly sweep: chance level (held-out prevalence) and AUC-PR of both scorers in both arms at the 7 chronological cuts, recomputed from the archived per-record scores with ECOD refit at each cut; "agree" marks the cuts at which the two arms order the scorers the same way. Figure 3 draws the timestamp-order columns.

	timestamp order			day round robin			
cut	chance	detector	ECOD	chance	detector	ECOD	agree
60%	0.350142	0.483383	0.451502	0.254153	0.395789	0.386564	yes
65%	0.400020	0.528435	0.498989	0.240532	0.402082	0.385961	yes
70%	0.461644	0.578913	0.561230	0.234708	0.436118	0.389094	yes
75%	0.429028	0.508857	0.556352	0.229687	0.477333	0.389087	no
80%	0.513066	0.585598	0.639969	0.241975	0.536930	0.400022	no
85%	0.682350	0.728355	0.758205	0.252396	0.544998	0.417522	no
90%	0.716250	0.799279	0.791671	0.285544	0.574244	0.474453	yes

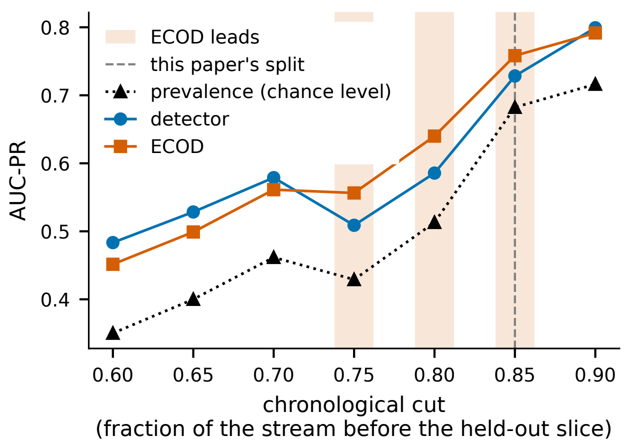

Fig. 3. AUC-PR of the two deterministic scorers at each of the 7 chronological cuts of the timestamp-ordered CICIDS2017 stream. The held-out prevalence at each cut is drawn on the same axis. It is the AUC-PR chance level. Shaded cuts are the 3 at which ECOD leads. The dashed line marks the cut this paper's fixed split uses. Values are recomputed from the archived per-record scores, as those of Table 8 are, with ECOD refit at each cut, and so differ slightly from Table 4 at the cut this paper uses.

leads in timestamp order and the detector leads in day round robin — and agree at the other 4, where the detector leads in both arms; the detector leads in the day-round-robin arm at every cut. The reversal is therefore a property of this region of cuts, not of every cut.

6.3 ECOD's scores depend on the batch they are scored in

The sweep above evaluates the same 240000 held-out records that Section 5.3 evaluates, under the same fitted model, and returns a different ECOD number. The discrepancy is not noise — ECOD is deterministic and both figures reproduce exactly on re-execution — and running it down exposes a property of the reference implementation that bears on every study that reports an ECOD number.

PyOD's `ECOD.fit` stores the training matrix, and `decision_function` then prepends it to whatever batch it is asked to score before recomputing the per-column empirical CDFs (PyOD 2.0.5, `pyod/models/ecod.py`):

```
if hasattr(self, 'X_train'):
    original_size = X.shape[0]
    X = np.concatenate((self.X_train, X), axis=0)
self.U_l = -1 * np.log(column_ecdf(X))
self.U_r = -1 * np.log(column_ecdf(-X))
skewness = np.sign(skew(X, axis=0))
...
decision_scores_ = self.O.sum(axis=1)[-original_size:]
```

The ECDF denominator is the height of the concatenated matrix and the skewness sign is taken over it, so a record's score is a function of which *other* records shared its `decision_function` call. ECOD is transductive over the scored batch, not a fitted function evaluated pointwise.

We measured the size of the effect with everything else pinned. The evaluated index set is fixed at the same 240000 records, the model is fitted once on the same benign training rows, and only the records accompanying them in the call vary, in number and therefore in composition. Scored alone, ECOD reaches $AP = 0.758205$; scored in the 480000-record validation-plus-test call this paper's contrast arms use, the same records give $AP = 0.755142$ — a difference of 0.003063 *AP* on identical records with an identical model. The full ladder has four rungs, each scoring those same 240000 records preceded by successively more of the validation records immediately before the split: $AP = 0.758205$ at 240000 records, 0.760029 at 300000, 0.762108 at 360000, and 0.755142 at 480000. The value therefore rises over the first three rungs and falls at the fourth, spanning 0.006966 *AP*: the dependence is not monotone in batch size and cannot be corrected for by a simple adjustment.

The second of those two values is the diagnostic one: scoring the held-out records inside the validation-plus-test batch reproduces, from an independent pass over the archived design matrix, the value the timestamp-ordered arm of Section 5.3 reports (0.755142), to the last reported decimal. The batch composition therefore accounts for the discrepancy completely rather than approximately, which is what distinguishes this from an unexplained numerical difference. The threshold inherits the dependence too, because it is read off the validation portion of the same call.

What is and is not new here. That ECOD mixes the training and scored sets is known: it was reported as a defect against the reference implementation in 2022 [36] and remains open, and ADBench [25] evaluates PyOD detectors in a fixed harness that holds the scored batch constant, which sidesteps rather than measures the issue. The ECOD paper [32] is not silent on new records: its Section IV-D states that ECOD requires no re-training to fit them, given a relatively large sample and no assumed data shift. What it does not state is a rule for scoring an out-of-sample record while leaving the fitted per-dimension ECDFs unchanged rather than recomputing them over the concatenation, and its scores are defined for the rows of the input matrix, so the implementation is not contradicting the specification: the specification is silent at exactly the point where the batch enters. It is not the only unresolved question about this reference: a separate open report [26] argues that the aggregation the code applies is not the one either the ECOD or the COPOD paper describes. We take no position on that second report; we note only that a detector used as a fixed point of comparison across a literature has open, unresolved questions about what it computes. What we add is narrow and precise: a *quantification* of the effect on a real benchmark with the evaluated records held identical, and the statement of its cross-study consequence. We searched for a published measurement of the effect's size and found none — the behaviour is recorded in the implementation's issue tracker and, so far as we can establish, has been treated as a correctness question rather than measured as an evaluation hazard — so we state this as a first quantification and a first statement of the cross-study consequence, in the ordinary sense that we are aware of no prior one. We would rather be corrected on priority than have the measurement go unreported, and the claim is about the measurement, not about the mechanism, which is not ours. That consequence is that published ECOD numbers are not comparable across studies that score a different batch, in size or composition, even on the same dataset, the same split and the same model, and that a reported ECOD number is therefore under-specified unless the scoring batch is reported with it. Campos et al. [12] showed for unsupervised outlier detection generally that reported comparisons depend on evaluation choices that papers rarely state.

We hold ourselves to that standard. Every ECOD number in this paper states the batch it was scored in. The contrast, LITNET and sweep numbers come from a call of the form

```
run_batch_reference('ecod', Xfit, np.vstack([Xva, Xte]))
```

so for those the scoring batch is validation-plus-test: 480000 records for both CICIDS arms (Section 5.3), 150000 for each per-capture LITNET stream and 450000 for the pooled composite (Section 5.4), and between 160917 and 480000 across the levels of the prevalence sweep (Section 8), which resamples the stream and so changes the batch with the level. The shared-record analysis of Section 6 reports ECOD under two batches: each arm's 240000-record held-out slice, restricted afterwards to the 78000 shared records, and the shared records scored as their own 78000-record batch; the split sweeps above score the tail of the stream at each cut, so their batch is the test slice itself and varies by cut. Cross-arm comparisons in this paper hold the batch fixed in size but not in composition: the two arms' validation-plus-test batches have the same 480000 records in count and different records in content, so equal size alone does not remove the batch dependence from the contrast of Section 5.3. Holding the timestamp arm's fitted model and the batch size fixed and changing only the batch content, by scoring the 78000 shared records inside the timestamp-order held-out slice and inside the day-round-robin held-out slice (240000 records each, differing in 162000), moves their AUC-PR from 0.844487 to 0.852039, a difference of 0.007552 *AP* (0.009096 AUC-ROC), so composition alone enters, as the mechanism implies: the empirical CDFs are recomputed over the concatenation, whose size and content both change every record's score. The batch effect measured on identical records, 0.003063 *AP* and at most 0.006966 *AP* across the ladder, is below the smallest reversal margin of that contrast, 0.026805 *AP* with interval lower bound 0.021099. Under its own 240000-record batch the timestamp-order ECOD scores 0.758205, a margin of 0.029868 *AP* over the detector, so ECOD leads that arm under both batches. Cross-arm ECOD comparisons are batch-controlled only where the rescoring of Table 6 makes them so, by scoring identical records as an identical batch under each arm's fitted model. The 240000-versus-480000 discrepancy is visible precisely because the split sweep and the contrast were the one pair of analyses that did not share a batch size.

7 The Deployed Score Ranks Worse Than Its Own Tail Term

The audit of Section 3 establishes what the two branches *are*. This section measures what they *do*, and the answer is a finding in its own right rather than a corollary: the way the two branches are combined destroys discriminative power that one of them already had.

The score is $\max(\text{tail}, 0.25 \cdot P(r \leq 5))$, and a branch's mean magnitude says nothing about which branch ranks. Scoring the timestamp-ordered held-out slice with each branch alone, using the components `update_score` itself computed:

Table 8. Branch-wise discrimination on the timestamp-ordered held-out slice. Values are recomputed from the archived per-record score dump and so differ from Table 4 by 0.000018 *AP* on the deployed composition (Section 6).

scoring branch	*AP*	AUC-ROC
deployed composition	0.728355	0.526623
tail term only	0.831832	0.829281
auxiliary term only	0.600270	0.281890

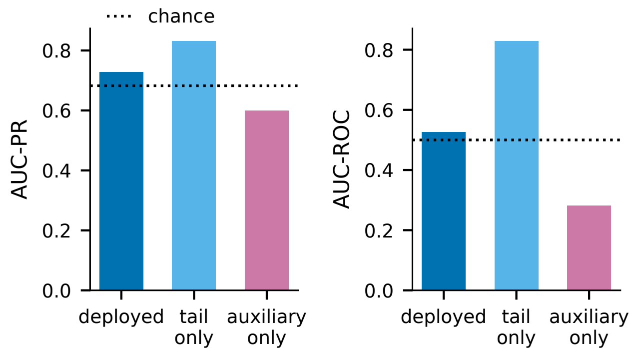

Fig. 4. Branch-wise discrimination on the timestamp-ordered held-out slice, the values of Table 8. Left: AUC-PR with the chance level at the slice prevalence, 0.682350. Right: AUC-ROC with the chance level of an uninformative ranking. The deployed composition is the detector of the other figures. The tail term alone ranks above the deployed composition on both metrics, and the auxiliary term ranks below chance on AUC-ROC.

The tail term alone *outranks* the deployed composition, by 0.103477 *AP* and 0.302658 AUC-ROC. Deleting the auxiliary branch would therefore improve the detector on this slice. The event-block bootstrap intervals of the three rows of Table 8 are [0.714449, 0.744732] *AP* and [0.510641, 0.542664] AUC-ROC for the deployed composition, [0.822025, 0.842969] and [0.824120, 0.834607] for the tail term alone, and [0.583380, 0.619628] and [0.272288, 0.291531] for the auxiliary term alone; the tail-only and deployed intervals do not overlap on either metric. Figure 4 draws the three branches against chance.

The mechanism is identified on this slice, and at the block lengths at which the branches' AUC-PR intervals separate (100 and 250 records; Section 12) its sampling uncertainty over the slice is small. The auxiliary term ranks *below* chance here (AUC-ROC 0.281890 against 0.5, with event-block bootstrap interval [0.272288, 0.291531], entirely below 0.5), which means it is not noise on this slice but an inverted signal: it scores attacks lower than benign records more often than not. Because the deployed score is a max, the auxiliary value sets the record's score whenever it exceeds the tail value — which is 58.452% of this held-out slice, counted from the same archived components — and the record's rank then follows from all scores together, so an inverted signal is written into the scores of a working one. That is why the deployed composition's AUC-ROC (0.526623) sits near chance while its tail component reaches 0.829281.

Three consequences follow, and we state them at their proper strength. First, the near-chance ranking of the deployed detector on this slice is a property of the *composition*, not of either component: a defect in how two signals are combined, not in either signal, and it would not be found by any evaluation that scores only the assembled detector. Second,

Table 9. AUC-PR by prevalence level on the assembled construction (mean over three resampling draws). “Chance level” is the achieved held-out prevalence. At the unresampled level all draws see the same records; at resampled levels the three draws of every method differ by the resampling draw. Per-draw values and normalized lift are in the archived findings.

Level	Chance level	evaluated detector	HST	LODA	ECOD (batch)	LOF (batch)
5%	0.049524	0.367164	0.199261	0.207581	0.166239	0.537596
10%	0.099004	0.447213	0.299702	0.247426	0.252198	0.697003
unresampled	0.252396	0.544998	0.433044	0.341715	0.418966	0.863194
40%	0.399995	0.494217	0.476031	0.399422	0.527364	0.896968
64%	0.639991	0.619546	0.499341	0.524691	0.678404	0.904855

it generalises as a warning rather than as a measurement — any detector that combines a calibrated signal with an uncalibrated one under a max (or any other order statistic that lets one branch dominate) can have the worse branch silently determine its ranking, and no downstream metric will attribute the loss. Third, it supersedes, in the opposite direction, any claim that the tail term simply performs the discriminative work of the deployed score: the tail term does not merely carry the score, it is held back by it. We state the measurement as a property of one slice of one benchmark; the branch-wise decomposition needed to find it costs nothing, because the components are values `update_score` already computes.

8 Prevalence Within One Assembled Construction

An earlier version reported a prevalence sweep as evidence about deployment regimes. Retained here, it is a controlled experiment *on the assembled (interleaved) CICIDS2017 stream* and nothing more: prevalence is varied by stratified resampling at five levels, three draws each. It does not isolate prevalence either — only the unresampled level’s draws see identical records, so which records are retained, how many of them there are, and how far apart in the stream the retained ones sit all vary with the nominal level. Table 9 reports AP with the achieved held-out prevalence as the chance level beside every prevalence level.

Two observations. The first is at the unresampled level, where all draws see identical records and the detector, ECOD and LOF are each deterministic, so a flat comparison is admissible here — identical records, no seed variance to withhold: the batch label-privileged reference LOF attains $AP = 0.863194$ against the detector’s $AP = 0.544998$ on that identical slice. Second, the detector’s additive lift $AP - p$ runs 0.317640 at 5%, 0.348209 at 10%, 0.292602 unresampled, 0.094222 at 40%, and -0.020445 at 64%, and its normalized form $(AP - p)/(1 - p)$ runs 0.334190, 0.386471, 0.391386, 0.157035 and -0.056790 over the same five levels — both below zero at the highest level, meaning below what an uninformative ranking achieves. The additive form peaks at the 10% level and the normalized form at the unresampled one, which is why both are reported. Nothing in this section transfers to the timestamp-ordered stream or to deployment.

9 An Alternative Reset Formulation

Section 3 showed the evaluated reset posterior is pinned below the cap by the published recursion. An alternative formulation places a prior-predictive term on the reset branch, so that the change point is modelled as occurring before x_t rather than after it and the reset hypothesis is informed by x_t. We implemented **one untuned instance** of that formulation — a Normal–Inverse-Gamma prior predictive, evaluated per feature and summed: a Student-t centred at the running global mean, with ν=2 degrees of freedom and squared scale $2\sigma^2$, where σ^2 is the running global variance floored at 10^{-4}. Its hyperparameters are κ_0=1, α_0=1 and β_0=σ^2, which give ν=$2\alpha_0$ and squared scale $(\beta_0/\alpha_0)(1 + 1/\kappa_0)$;

Table 10. One untuned alternative formulation saturates the score. Diagnostics over the first 15000 records of the ablation stream; detection metrics on the 200000-record prefix (held-out 30000 records, 4840 attacks, chance level 0.161333).

	evaluated detector	one untuned variant
mean posterior mass $P(r \le 5)$	0.006472	1
scores exactly at the 0.25 cap	0.004733	0.9274
distinct score values	3899	747
score standard deviation	0.218999	0.159341
AP on the prefix	0.397486	0.169912
normalized lift $(AP - p)/(1 - p)$	0.281581	0.010229
AUC-ROC on the prefix	0.848216	0.509620

nothing is tuned — and measured both variants under a 30-minute compute cap whose reductions bind this paragraph: one stream (udp_flood), a 200000-record prefix, one seed. On the synthetic probe the variant responds: $P(r=0)$ peaks at 1 after a 6σ shift against 0.001 for the evaluated detector. On real data it ranks near chance: $AP = 0.169912$ against 0.397486 (chance level 0.161333; normalized lift 0.010229 against 0.281581), with AUC-ROC 0.509620.

The prefix diagnostics of Table 10 are consistent with that ranking, not proof of it: they are measured on the first 15000 records of the stream, while AP and AUC-ROC are measured on the final 30000 held-out records of the 200000-record prefix, a different population, and the per-record scores of that held-out slice were not archived, so no equality of the two score distributions is shown. Under the variant the posterior sits on short runs at every step (mean $P(r \le 5) = 1$), the $0.25 \cdot P(r \le 5)$ branch saturates, 0.9274 of scores sit exactly at the 0.25 cap, and 747 distinct values are emitted against 3899. The score is *not* constant — it takes many values and has standard deviation 0.159341 — but it ranks near chance on this sample, which is what an AUC-ROC close to 0.5 indicates. The two variants are degenerate in different quantities, and we state each in the quantity measured. Below the cap the evaluated variant's reset posterior is pinned at the hazard rate and carries no information from the current record (Section 3); under this alternative it is the short-run mass that saturates, at $P(r \le 5) = 1$, and the score sits at the 0.25 cap on 0.9274 of records. The archived run recorded $P(r=0)$ for this alternative only on the synthetic probe, so we assert nothing here about how often it resets on the stream. We draw no conclusion about what a *tuned* prior-predictive scale would achieve; that requires a training/validation tuning protocol and more than one stream, and selecting it on test labels is forbidden by our protocol.

10 Provenance Discipline

Every measured value in this paper is a LaTeX macro generated from archived run manifests recording git commit, configuration hash, input SHA-256, seed, environment hash, timestamps and output paths. Protocol constants that define the experiment rather than result from it (the 70/15/15 split, budget sizes, the auxiliary weight, library versions) and values quoted from cited third-party work are typed, and are identified as such where they appear. A claim ledger maps every sentence of the abstract and introduction to its generating run.

What the checks cover, stated as a scope rather than a name. The build fails on: a measurement-shaped number with no manifested value, anywhere in the manuscript, in any file it includes, or in any file the claim ledger cites; a macro two runs claim with different values; drift between the derived macro index and the manifests behind it; a cited key with no bibliography entry; and a derived quantity that disagrees with the values printed beside it. Files whose purpose is to record withdrawn numbers are scanned and reported rather than failed, because forcing a withdrawn value through the macro layer would manifest an error as a measurement, and the few genuine non-measurements inside enforced files sit in a named allowlist with a stated reason each.

The checks do *not* cover: protocol constants and values quoted from cited work, which are typed and identified as such; prose; or — the important one — whether the code, labels, preprocessing or interpretation are correct. Provenance establishes that a displayed number came from a recorded execution on recorded inputs, and nothing beyond that.

We state the covered set explicitly, rather than the mechanism's name, for a reason that generalises past this paper: a check is trusted according to what its name claims and enforces only what it inspects, and the gap between the two is invisible precisely because the name is reassuring — a gate reported that every number in this manuscript traced to a manifest while reading only the file it had itself generated from those manifests, and a compile check reported zero undefined references in a document carrying citations that resolved to nothing. Both were true statements about a narrower scope than their names implied. A reader should ask of any such check what set it actually reads.

11 Relationship to Prior Versions of This Work

Earlier versions of this manuscript were publicly posted and are superseded by this one. Because a per-result-group account of what is reused, re-derived, corrected, withdrawn or new would identify those versions — and they are not anonymous — that account is supplied to the editors confidentially rather than printed here, together with the dated version history and the list of claims each correction invalidates.

One statement can be made in the body without breaking anonymity, and is made here because it bears on how the results should be read. Some measurements in this paper also appear in the earlier versions, reported there under an interpretation that this paper withdraws and replaces. Which measurements those are, which results are new, and which earlier results are withdrawn is as follows. The pooled LITNET composite and the assembled CICIDS arm are the same measurements as in v1 and v2, reported there under a regime interpretation that this paper replaces with an assembly interpretation; the timestamp-ordered CICIDS arm, the per-capture LITNET streams, the shared-record analysis and the method-identity audit have no counterpart in any earlier version; and results on a third dataset that appeared in the earlier versions are withdrawn and are not relied upon anywhere in this paper.

The corrected-incident log in the artifact records each correction with its evidence and closure status. The count of incidents is not itself a quality claim.

12 Threats to Validity

Internal. The assembly treatment is uncontrolled by construction: it moves held-out membership, prevalence and order together, so no single factor is identified by the contrast alone. Section 6 removes the membership difference post hoc and finds the ordering reversal does not survive, which attributes the effect to sample membership under the assumption stated there, while the detector's prefix and ECOD's training rows still differ between the arms; it does not decompose the remaining factors, and a factorial design still would. ECOD is a label-privileged diagnostic reference, not a competitor under equal information. Determinism removes seed variance only. With attack runs of median/p90/max 2/70/2522 records an IID row bootstrap would be inappropriate, so every interval this paper reports comes from a moving-block bootstrap [28] over the archived per-record scores of the evaluated slice, in stream order: block length 100 records, longer than the p90 run, 1000 resamples, and the 2.5th and 97.5th percentiles of the resampled statistic. Those intervals quantify sampling uncertainty over the evaluated slice only, not over the window selection or the subsample, and a rolling-origin procedure is not applied. Rerunning that bootstrap at block lengths 250 and 2600 records, the second longer than the longest attack run, changes exactly one conclusion: every margin interval stays above zero and the auxiliary-only AUC-ROC interval stays below 0.5 at both lengths, but at 2600 records the tail-only and deployed AUC-PR intervals, [0.781947, 0.880632] and [0.653543, 0.807028], overlap while the AUC-ROC intervals stay disjoint (0 of nine checks change at the shorter length and 1 at the longer; the rerun is archived in the artifact). Intervals are reported for the two margins of Section 5.3, the margins of Table 6 and the branch values of Table 8; the LITNET per-stream values of Table 5, the per-cut values of Table 7, the prevalence-level means of Table 9 and the one-stream result of Section 9 carry none, because the per-record scores behind the LITNET, sweep and Section 9 results were not archived and the per-cut values are single recomputations, so those results have weaker inferential support than the headline contrast. Stochastic baselines appear either as single draws marked as such or as means over draws with the three draws stated, and the per-draw values are archived.

External. Two benchmarks, one split rule, one corpus per benchmark. The CICIDS mechanism depends on that dataset's scripted per-day schedule meeting a positional split; it does not establish the direction or magnitude of assembly effects elsewhere. The LITNET identity depends on equal budgets and a divisible boundary. The corpus is a 76.628% budgeted subsample. The timestamp-ordered held-out slice is one 204.2-minute window — what a positional chronological split yields on this corpus, stated rather than engineered around. Timestamp order is the least transformed reference available for CICIDS2017, not a universal gold standard: multi-sensor deployments legitimately merge asynchronous sources, and the durable recommendation is to preserve capture and time metadata, justify the assembly, and report sensitivity to plausible alternatives.

Construct. We do not claim the evaluated detector is change-point detection, we do not claim the untuned variant characterizes change-point detection with an informed reset, and we do not claim that ordering alone causes the outcome change.

13 Conclusion

On these two benchmarks, the step that assembles capture files into an evaluation stream is an uncontrolled experimental treatment: it changes which records are evaluated, at what prevalence, in what order, and thereby what is measured. On CICIDS2017 the same multiset reordered yields held-out slices sharing 32.5% of their records at prevalences 42.9954 points apart, and reverses the measured ordering of the two deterministic scorers — a reversal that disappears once both arms are restricted to the records they share, attributing it, under the assumption that history contributes no more on the unshared records than on the shared ones, to the sample the assembly selects rather than to the order it imposes. That restriction does not bound history's effect on comparisons between shared and unshared records, which the full-slice value also contains. On LITNET-2020, pooling reports an operating point no capture exhibits. The practical recommendation is not that chronology is always right; it is that assembly belongs in the experimental design and in the provenance record — report capture identity, mixture weights, split membership, held-out overlap, and sensitivity to the plausible alternatives — so that a reported operating point can be recognized as a property of the assembly when it is one.

Data and Artifact Availability

The code, stream-assembly scripts with SHA-256 verification, all run manifests, the macro layer, the claim ledger and the corrected-incident history are archived. The repository is public at https://github.com/MichelYsf/rcbsid-paper (branch `rebuild/honest-v1`); the artifact is archived as version 2.2.0 of the Zenodo record lineage, doi:10.5281/zenodo.22673735, which supersedes version 2.1.0 (doi:10.5281/zenodo.22638195), version 2.0.0 (doi:10.5281/zenodo.22213264) and version 1.0.0 (doi:10.5281/zenodo.20074590).

Acknowledgements: Generative AI Usage

In accordance with the ACM Policy on Authorship, the authors disclose that generative AI tools were used in producing this work. Large language models were used, under continuous human direction, to write and refactor analysis scripts, to draft prose subsequently verified against the archived measurements, and to operate an adversarial review harness that audited the authors' own claims. No generative AI tool is an author of this work, and none produced any reported measurement: every measured value was generated by executed code that wrote an archived run manifest in the same execution, and the provenance gate and claim ledger exist in part to make that boundary externally auditable. The authors take full responsibility for the entire content of this work, including any portion drafted with such assistance.

Companion Manuscript Disclosure

A companion manuscript from the same research programme (arXiv:2510.09619) shares parts of the underlying codebase and predates the audit reported here. The method-identity findings of Section 3 apply to that shared lineage, and correcting it is in hand. It is a public preprint and is not under review at any venue, and there is accordingly no other editor to inform and no concurrent consideration to declare. Its relationship to the results reported here, and the relationship to the earlier public versions of this manuscript, are stated in Section 11; the earlier public versions are arXiv:2605.24696 v1 and v2.

References

[1] Ryan Prescott Adams and David J. C. MacKay. 2007. Bayesian Online Changepoint Detection. *arXiv preprint arXiv:0710.3742* (2007). doi:10.48550/arXiv.0710.3742

[2] Giovanni Apruzzese, Pavel Laskov, and Johannes Schneider. 2023. SoK: Pragmatic Assessment of Machine Learning for Network Intrusion Detection. In *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 592–614. doi:10.1109/EuroSP57164.2023.00042

[3] Giovanni Apruzzese, Luca Pajola, and Mauro Conti. 2022. The Cross-Evaluation of Machine Learning-Based Network Intrusion Detection Systems. *IEEE Transactions on Network and Service Management* 19, 4 (2022), 5152–5169. doi:10.1109/TNSM.2022.3157344

[4] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. 2022. Dos and Don'ts of Machine Learning in Computer Security. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 3971–3988.

[5] Stefan Axelsson. 2000. The Base-Rate Fallacy and the Difficulty of Intrusion Detection. *ACM Transactions on Information and System Security* 3, 3 (2000), 186–205. doi:10.1145/357830.357849

[6] Rina Foygel Barber, Emmanuel J. Candès, Aaditya Ramdas, and Ryan J. Tibshirani. 2023. Conformal Prediction Beyond Exchangeability. *The Annals of Statistics* 51, 2 (2023), 816–845. doi:10.1214/23-AOS2276

[7] Seth Barrett, Gokila Dorai, Lin Li, and Swarnamugi Rajaganapathy. 2026. FADES: Adaptive Drift Estimation via Conformal Signals for Streaming Intrusion Detection. *Electronics* 15, 10 (2026), 2114. doi:10.3390/electronics15102114

[8] Seth Barrett, Lin Li, Gokila Dorai, and Swarnamugi Rajaganapathy. 2026. FIRCE: A Framework for Intrusion Response and Conformal Evaluation. In *2026 International Conference on Smart Applications, Communications and Networking (SmartNets)*. 1–9. arXiv:2605.01962. doi:10.1109/SMARTNETS69662.2026.11604738

[9] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy (Eds.). 2016. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.

[10] Md Anas Biswas. 2026. The Evaluation Protocol Is the Hidden Variable: A Drift-Aware Re-Examination of Flow-Based Intrusion Detection Benchmarks. *Preprints.org* (2026). Preprint, posted 11 June 2026. doi:10.20944/preprints202606.0903.v1

[11] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. 2000. LOF: Identifying Density-Based Local Outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. 93–104. doi:10.1145/342009.335388

[12] Guilherme O. Campos, Arthur Zimek, Jörg Sander, Ricardo J. G. B. Campello, Barbora Micenková, Erich Schubert, Ira Assent, and Michael E. Houle. 2016. On the Evaluation of Unsupervised Outlier Detection: Measures, Datasets, and an Empirical Study. *Data Mining and Knowledge Discovery* 30, 4 (2016), 891–927. doi:10.1007/s10618-015-0444-8

[13] Yang Cao, Yixiao Ma, Ye Zhu, and Kai Ming Ting. 2025. Revisiting Streaming Anomaly Detection: Benchmark and Evaluation. *Artificial Intelligence Review* 58, 1 (2025), 8. doi:10.1007/s10462-024-10995-w

[14] Marta Catillo, Andrea Del Vecchio, Antonio Pecchia, and Umberto Villano. 2021. A Critique on the Use of Machine Learning on Public Datasets for Intrusion Detection. In *Quality of Information and Communications Technology (QUATIC 2021) (Communications in Computer and Information*

Science, Vol. 1439). Springer, 253–266. doi:10.1007/978-3-030-85347-1_19

[15] Marta Catillo, Antonio Pecchia, and Umberto Villano. 2023. Machine Learning on Public Intrusion Datasets: Academic Hype or Concrete Advances in NIDS?. In *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks, Supplemental Volume (DSN-S)*. 132–136. doi:10.1109/DSN-S58398.2023.00038

[16] Robertas Damasevicius, Algimantas Venckauskas, Sarunas Grigaliunas, Jevgenijus Toldinas, Nerijus Morkevicius, Tautvydas Aleliunas, and Paulius Smuikys. 2020. LITNET-2020: An Annotated Real-World Network Flow Dataset for Network Intrusion Detection. *Electronics* 9, 5 (2020), 800. doi:10.3390/electronics9050800

[17] Jesse Davis and Mark Goadrich. 2006. The Relationship Between Precision-Recall and ROC Curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*. 233–240. doi:10.1145/1143844.1143874

[18] Zhiguo Ding and Minrui Fei. 2013. An Anomaly Detection Approach Based on Isolation Forest Algorithm for Streaming Data Using Sliding Window. *IFAC Proceedings Volumes* 46, 20 (2013), 12–17. doi:10.3182/20130902-3-CN-3020.00044

[19] Charles Elkan. 2001. The Foundations of Cost-Sensitive Learning. In *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence (IJCAI 2001)*. Morgan Kaufmann, 973–978.

[20] Gints Engelen, Vera Rimmer, and Wouter Joosen. 2021. Troubleshooting an Intrusion Detection Dataset: The CICIDS2017 Case Study. In *2021 IEEE Security and Privacy Workshops (SPW)*. 7–12. doi:10.1109/SPW53761.2021.00009

[21] Paul Fearnhead and Zhen Liu. 2007. On-Line Inference for Multiple Changepoint Problems. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 69, 4 (2007), 589–605. doi:10.1111/j.1467-9868.2007.00601.x

[22] João Gama, Raquel Sebastião, and Pedro Pereira Rodrigues. 2013. On Evaluating Stream Learning Algorithms. *Machine Learning* 90, 3 (2013), 317–346. doi:10.1007/s10994-012-5320-9

[23] Sudipto Guha, Nina Mishra, Gourav Roy, and Okke Schrijvers. 2016. Robust Random Cut Forest Based Anomaly Detection on Streams. In *Proceedings of the 33rd International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 48)*. PMLR, 2712–2721.

[24] Ambarish Gurjar and L. Jean Camp. 2026. Predicting Tail-Risk Escalation in IDS Alert Time Series. *arXiv preprint arXiv:2601.14299* (2026).

[25] Songqiao Han, Xiyang Hu, Hailiang Huang, Minqi Jiang, and Yue Zhao. 2022. ADBench: Anomaly Detection Benchmark. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*. 32142–32159. arXiv:2206.09426. doi:10.52202/068431-2329

[26] Hellsice. 2024. ECOD and COPOD Decision Functions Switched. PyOD issue #561, https://github.com/yzhao062/pyod/issues/561. Opened 30 April 2024; open as of 3 September 2026.

[27] Jeremias Knoblauch and Theodoros Damoulas. 2018. Spatio-Temporal Bayesian On-Line Changepoint Detection with Model Selection. In *Proceedings of the 35th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 80)*. PMLR, 2718–2727. arXiv:1805.05383.

[28] Hans R. Künsch. 1989. The Jackknife and the Bootstrap for General Stationary Observations. *The Annals of Statistics* 17, 3 (1989), 1217–1241. doi:10.1214/aos/1176347265

[29] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé, and Éric Totel. 2023. Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes. In *Risks and Security of Internet and Systems (CRiSIS 2022) (Lecture Notes in Computer Science, Vol. 13857)*. Springer, 18–33. doi:10.1007/978-3-031-31108-6_2

[30] Rikard Laxhammar and Göran Falkman. 2014. Online Learning and Sequential Anomaly Detection in Trajectories. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 6 (2014), 1158–1173. doi:10.1109/TPAMI.2013.172

[31] Zheng Li, Yue Zhao, Nicola Botta, Cezar Ionescu, and Xiyang Hu. 2020. COPOD: Copula-Based Outlier Detection. In *2020 IEEE International Conference on Data Mining (ICDM)*. 1118–1123. doi:10.1109/ICDM50108.2020.00135

[32] Zheng Li, Yue Zhao, Xiyang Hu, Nicola Botta, Cezar Ionescu, and George H. Chen. 2023. ECOD: Unsupervised Outlier Detection Using Empirical Cumulative Distribution Functions. *IEEE Transactions on Knowledge and Data Engineering* 35, 12 (2023), 12181–12193. doi:10.1109/TKDE.2022.3159580

[33] Lisa Liu, Gints Engelen, Timothy Lynar, Daryl Essam, and Wouter Joosen. 2022. Error Prevalence in NIDS Datasets: A Case Study on CIC-IDS-2017 and CSE-CIC-IDS-2018. In *2022 IEEE Conference on Communications and Network Security (CNS)*. IEEE, 254–262. doi:10.1109/CNS56114.2022.9947235

[34] Tetiana Manzhos, Tetiana Ianevych, and Olga Melnyk. 2026. Average Precision at Cutoff k under Random Rankings: Expectation and Variance. *Modern Stochastics: Theory and Applications* 13, 3 (2026), 357–374. arXiv:2511.02571. doi:10.15559/26-VMSTA298

[35] Emaad Manzoor, Hemank Lamba, and Leman Akoglu. 2018. xStream: Outlier Detection in Feature-Evolving Data Streams. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*. 1963–1972. doi:10.1145/3219819.3220107

[36] mbongaerts. 2022. ECOD Mixes Train Set and Test Set. PyOD issue #401, https://github.com/yzhao062/pyod/issues/401. Opened 28 April 2022; open as of 3 September 2026.

[37] Matthew B. McDermott, Haoran Zhang, Lasse Hyldig Hansen, Giovanni Angelotti, and Jack Gallifant. 2024. A Closer Look at AUROC and AUPRC under Class Imbalance. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. 44102–44163. arXiv:2401.06091. doi:10.52202/079017-1400

[38] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. In *Proceedings of the 2018 Network and Distributed System Security Symposium (NDSS)*. Internet Society. arXiv:1802.09089. doi:10.14722/ndss.2018.23204

[39] Zach Moczkodan and Hany Ragab. 2026. Do Transformers Actually Help Intrusion Detection? A Temporal Sequence Evaluation on CIC-IDS2017. *arXiv preprint arXiv:2606.11098* (2026).

[40] Daniel Olszewski, Allison Lu, Carson Stillman, Kevin Warren, Cole Kitroser, Alejandro Pascual, Divyajyoti Ukirde, Kevin Butler, and Patrick Traynor. 2023. "Get in Researchers; We're Measuring Reproducibility": A Reproducibility Study of Machine Learning Papers in Tier 1 Security Conferences. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 3433–3459. doi:10.1145/3576915.3623130

[41] Feargus Pendlebury, Fabio Pierazzi, Roberto Jordaney, Johannes Kinder, and Lorenzo Cavallaro. 2019. TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time. In *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, 729–746.

[42] Tomáš Pevný. 2016. Loda: Lightweight On-Line Detector of Anomalies. *Machine Learning* 102, 2 (2016), 275–304. doi:10.1007/s10994-015-5521-0

[43] Markus Ring, Sarah Wunderlich, Deniz Scheuring, Dieter Landes, and Andreas Hotho. 2019. A Survey of Network-Based Intrusion Detection Data Sets. *Computers & Security* 86 (2019), 147–167. doi:10.1016/j.cose.2019.06.005

[44] Takaya Saito and Marc Rehmsmeier. 2015. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE* 10, 3 (2015), e0118432. doi:10.1371/journal.pone.0118432

[45] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. 108–116. doi:10.5220/0006639801080116

[46] Robin Sommer and Vern Paxson. 2010. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. In *2010 IEEE Symposium on Security and Privacy*. IEEE, 305–316. doi:10.1109/SP.2010.25

[47] Swee Chuan Tan, Kai Ming Ting, and Fei Tony Liu. 2011. Fast Anomaly Detection for Streaming Data. In *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence (IJCAI 2011)*. AAAI Press, 1511–1516. doi:10.5591/978-1-57735-516-8/IJCAI11-254

[48] Nesime Tatbul, Tae Jun Lee, Stan Zdonik, Mejbah Alam, and Justin Gottschlich. 2018. Precision and Recall for Time Series. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*. 1924–1934. arXiv:1803.03639.

[49] Gerrit J. J. van den Burg and Christopher K. I. Williams. 2020. An Evaluation of Change Point Detection Algorithms. *arXiv preprint arXiv:2003.06222* (2020).

[50] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. 2005. *Algorithmic Learning in a Random World*. Springer, New York. doi:10.1007/b106715

[51] Renjie Wu and Eamonn J. Keogh. 2023. Current Time Series Anomaly Detection Benchmarks Are Flawed and Are Creating the Illusion of Progress. *IEEE Transactions on Knowledge and Data Engineering* 35, 3 (2023), 2421–2429. doi:10.1109/TKDE.2021.3112126